

A REPORT
ON
GRAPH-BASED CONTEXT-AWARE OBJECT DETECTION FOR
AUTONOMOUS DRIVING IN COMPLEX DRIVING SCENES

BY
ANAGHA K M
ID No.: 2024AA05551

AT
Cognizant Technology Solutions, Bangalore

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
(Jul 2026)

A REPORT
ON
GRAPH-BASED CONTEXT-AWARE OBJECT DETECTION FOR
AUTONOMOUS DRIVING IN COMPLEX DRIVING SCENES

BY
ANAGHA K M
ID No.: 2024AA05551

Prepared in partial fulfilment of the
M.Tech in Artificial Intelligence and Machine Learning

AT
Cognizant Technology Solutions, Bangalore

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

(Jul 2026)

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
(RAJASTHAN)
WILP Division

Organization: Cognizant Technology Solutions **Location:** Bangalore, India

Duration: 14 weeks

Date of Start: 25th Apr, 2026

Date of Submission: 28th Jul, 2026

Title of the Project: Graph-Based Context-Aware Object Detection For Autonomous Driving In Complex Driving Scenes

ID No. / Name of the student: 2024AA05551 - Anagha K M

Supervisor: A K Sunitha, Senior Associate, Cognizant Technology Solutions, Bangalore, India

Additional Examiner: Visal Vidhya, Senior Associate, Cognizant Technology Solutions, Bangalore, India

Faculty Mentor: Ashwin Ganesan

Key Words: Object Detection; Autonomous Driving; YOLOv8; Graph Neural Network; Graph Convolutional Network; Scene Graph; Context-Aware Detection; Confidence Refinement; BDD100K; False-Positive Suppression

Project Areas: Computer Vision; Deep Learning; Graph Neural Networks; Autonomous Driving; Intelligent Perception Systems

Abstract:

Reliable object detection is essential for autonomous driving, where perception errors can affect downstream navigation decisions. Standard detectors such as YOLO provide fast detection, but their final confidence scores do not explicitly model relationships between nearby objects. This dissertation develops a graph-based post-detection refinement framework that keeps YOLOv8n frozen and represents each detection as a scene-graph node using geometry, confidence, and class-related features. A Graph Convolutional Network is trained with BDD100K ground-truth IoU labels to produce confidence corrections for YOLO detections. The final system is evaluated on a curated 500-image BDD100K subset using five-fold cross-validation, hard-scene analysis, and false-positive suppression diagnostics. Results show that AP@0.5 and mAP@0.5 remain close to the YOLO baseline, while selected threshold-level and hard-scene results show small F1 gains. The learned GCN also demonstrates genuine false-positive suppression behavior.

Signature of the Student

Name: Anagha K M

Date: 28-07-2026

Place: Bangalore, India

Signature of the Supervisor

Name: A K Sunitha

Date: 28-07-2026

Place: Bangalore, India

ACKNOWLEDGEMENTS

I would like to thank Cognizant Technology Solutions, Bangalore, for the opportunity and environment to carry out this dissertation as part of the Work Integrated Learning Programme.

I acknowledge A K Sunitha, Senior Associate, Cognizant Technology Solutions, who was the supervisor of record for this dissertation, and Visal Vidhya, Senior Associate, Cognizant Technology Solutions, who served as the additional examiner.

I am sincerely grateful to my faculty mentor, Ashwin Ganesan, for his guidance and feedback throughout this dissertation. His review of the mid-semester submission was directly responsible for several of the improvements in the final work, including the move from pseudo-label to ground-truth IoU-based evaluation, and the addition of hard-scene analysis and false-positive suppression diagnostics.

I thank BITS Pilani, Work Integrated Learning Programmes Division, for the academic framework and coursework that made this dissertation possible.

I gratefully acknowledge the open-source research and software community. This work was made possible by publicly available tools and libraries, including Python, PyTorch, PyTorch Geometric, Ultralytics YOLO, scikit-learn, pandas, NumPy, and matplotlib, and by the Berkeley DeepDrive team for making the BDD100K dataset publicly available for research use.

Finally, I thank my family and friends for their support throughout this dissertation.

Anagha K M

BITS ID: 2024AA05551

M.Tech in Artificial Intelligence & Machine Learning

Bangalore, India

July 2026

PREFACE – RESPONSE TO MID-SEMESTER EVALUATOR FEEDBACK

During the mid-semester evaluation, the main concern raised was that the early GCN training setup used pseudo-labels derived from YOLO confidence scores. This created a circular training setup because the same YOLO confidence values that the GCN was expected to refine were also being used to create the training labels. The evaluator also suggested that the final work should include ground-truth-based evaluation, hard-scene analysis, and evidence that the refinement module can suppress false positives rather than only boost confidence.

These points were addressed in the final implementation. First, the pseudo-label setup was replaced with ground-truth IoU-based supervision using BDD100K annotations. A detection was treated as a true positive only when it matched a ground-truth object of the corresponding class with IoU greater than or equal to 0.5. This removed the circularity present in the mid-semester version.

Second, the experiment was scaled to a curated 500-image BDD100K subset. Out of these images, 499 produced valid graph samples after detection filtering and graph construction. This larger evaluation made the cross-validation results more stable than the earlier small-run experiment.

Third, the final evaluation includes ablation studies comparing the YOLO baseline, YOLO + GCN learned refinement, YOLO + semantic rule refinement, and YOLO + GCN + semantic rule refinement. This separates the effect of the learned GCN component from the hand-designed semantic rule component.

Fourth, hard-scene subset analysis was added for categories such as dense traffic, pedestrian-vehicle interaction, occlusion-like scenes, night/low-visibility scenes, and traffic-control scenes. This was included because aggregate metrics alone may hide behavior in difficult driving conditions.

Finally, false-positive suppression diagnostics and failure-case visualizations were added. These results show that the learned GCN can produce negative confidence corrections for false-positive detections, while also revealing cases where semantic boosting can increase false positives. The final report therefore presents both strengths and limitations of the method.

Contents

Chapter 1: Introduction	1
1.1 Background and Motivation.....	1
1.2 Problem Statement	1
1.3 Aim and Objectives.....	2
1.4 Scope of Work.....	2
1.5 Contributions.....	3
1.6 Organization of the Report.....	3
Chapter 2: Literature Review.....	4
2.1 Overview	4
2.2 Object Detection in Autonomous Driving.....	4
2.3 YOLO-Based Object Detection.....	5
2.4 Transformer-Based Object Detection.....	5
2.5 Graph Neural Networks	5
2.6 Scene Graphs and Context-Aware Perception	6
2.7 Semantic Context in Object Detection	7
2.8 Research Gap.....	7
2.9 Summary	8
Chapter 3: Methodology.....	9
Chapter 4: Implementation.....	14
Chapter 5: Results and Discussion	17
Chapter 6: Conclusion and Future Work.....	24
Appendices.....	26
References.....	27
Glossary	29
Final Checklist.....	30

LIST OF TABLES

Table	Title	Page
Table 2.1	Positioning of the Proposed Work Relative to Related Methods	7
Table 3.1	Ablation Variants	10
Table 3.2	Semantic Context Examples	11
Table 3.3	Curated 500-Image Dataset Distribution	12
Table 4.1	Development Environment	14
Table 4.2	Output Artifacts	16
Table 4.3	Final System Configuration Summary	16
Table 5.1	Curated 500-Image Dataset Profile	17
Table 5.2	Detection and Graph Labeling Summary	17
Table 5.3	Five-Fold Cross-Validation Results	17
Table 5.4	Pooled Out-of-Fold AP@0.5 Results	18
Table 5.5	Fixed-Threshold F1 Results	19
Table 5.6	Hard-Scene F1 Improvements	19
Table 5.7	False-Positive Suppression Diagnostic	19
Table 5.8	Failure-Case Visualization Categories	21
Table 5.9	Evaluator Feedback Resolution	22

LIST OF FIGURES

Figure	Title	Page
Figure 3.1	End-to-End YOLO-to-Graph-to-GCN Refinement Pipeline	9
Figure 5.1	Train vs Validation BCE Loss Curves Across Five Folds (Regularized GCN)	18
Figure 5.2	Qualitative Comparison of YOLO Baseline and GT-Supervised Refined Detections – Example A	20
Figure 5.3	Qualitative Comparison of YOLO Baseline and GT-Supervised Refined Detections – Example B	20
Figure 5.4	Borderline Detection Crossing the Confidence Threshold After Refinement	21
Figure 5.5	Example of True-Positive Confidence Boosting (Report-Safe Case)	21

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ADAS	Advanced Driver Assistance System
AI	Artificial Intelligence
AP	Average Precision
BCE	Binary Cross Entropy
BDD100K	Berkeley DeepDrive 100K
CNN	Convolutional Neural Network
CV	Computer Vision / Cross-Validation
DETR	Detection Transformer
DL	Deep Learning
FN	False Negative
FP	False Positive
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GNN	Graph Neural Network
IoU	Intersection over Union
kNN	k-Nearest Neighbour
mAP	mean Average Precision
ML	Machine Learning
NMS	Non-Maximum Suppression
ROI	Region of Interest
TP	True Positive
YOLO	You Only Look Once

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Autonomous driving systems depend heavily on reliable perception. Before a vehicle can plan a safe path, apply braking, maintain lane position, or respond to traffic signals, it must first identify the surrounding objects correctly. These objects may include vehicles, pedestrians, cyclists, traffic lights, traffic signs, and other road users. A mistake at the perception stage can affect every downstream decision in the driving pipeline.

Object detection has improved significantly with deep learning. Detectors such as the YOLO family [1] are widely used because they provide fast detection in a single forward pass. As a result, they are well suited to applications that demand real-time throughput. Transformer-based detectors such as DETR [2] have likewise achieved competitive results by treating detection as a set-prediction task. However, even when these models perform well, their confidence scores are mainly assigned at the object level. They do not explicitly represent the relationship between detected objects.

This limitation is important in driving scenes. A pedestrian standing alone and a pedestrian standing near a moving vehicle may have similar visual appearance, but the surrounding context is very different. Similarly, a traffic light near multiple vehicles is more meaningful than an isolated signal-like detection in the background. A detector may implicitly learn some context through image features, but this context is not available as an explicit relational structure.

Graph Neural Networks offer a suitable formalism for capturing these inter-object relationships [3]. In a graph-based representation, each detected object can be treated as a node, and relationships between nearby objects can be represented as edges. As a result, the model can account for an object's surrounding context rather than treating each detection in isolation.

This dissertation investigates a lightweight post-detection refinement framework. Instead of modifying the YOLO detector, the proposed system leaves YOLO unchanged and applies a Graph Convolutional Network as a downstream module that adjusts its confidence scores. The intent is not to replace YOLO, but to examine whether contextual graph reasoning can improve or explain confidence refinement in complex driving scenes.

1.2 Problem Statement

Modern object detectors are effective at locating objects in images, but their confidence scores are not explicitly conditioned on structured inter-object relationships. In autonomous driving, this is a limitation because object importance and detection reliability often depend on surrounding scene context.

The problem addressed in this dissertation is: How can detections produced by a frozen YOLO model be restructured as a graph and adjusted through GNN-based contextual reasoning, while leaving the detector architecture untouched?

The work focuses specifically on confidence refinement. Bounding box coordinates and class labels are preserved from YOLO. The GCN only produces a confidence correction for each detection. This keeps the proposed system modular and lightweight.

1.3 Aim and Objectives

This dissertation sets out to design, build, and assess a context-aware refinement framework, built on graph representations, for object detection in autonomous-driving scenarios.

1. To study object detection methods used in autonomous driving and identify the limitation of independent object-level confidence scoring.
2. To implement a YOLOv8n-based baseline detector for driving scene images.
3. To transform YOLO detections into graph-structured data, with each detection forming a node and spatial or semantic relationships forming edges.
4. To train a Graph Convolutional Network for confidence refinement using ground-truth IoU-based labels.
5. To compare the YOLO baseline against YOLO + GCN learned refinement, YOLO + semantic rule refinement, and YOLO + GCN + semantic rule refinement.
6. To assess the system through AP@0.5, mAP@0.5, precision, recall, F1-score, subset-level performance on hard scenes, and false-positive-suppression diagnostics.
7. To analyze both successful and failure cases through qualitative visualization.

1.4 Scope of Work

This dissertation's scope is confined to object detection on individual, static driving-scene images. YOLOv8n is used as a frozen baseline, with graph-based confidence refinement layered on afterward as a post-processing step.

- YOLO-based object detection on BDD100K [4] driving images.
- Filtering of detections to driving-relevant classes.
- Scene graph construction from detections.
- Node feature generation using normalized bounding box geometry, confidence, and class-related information.
- Ground-truth IoU-based labeling of detections.

- GCN-based learned confidence refinement.
- Semantic rule-based refinement as an ablation.
- Five-fold image-level cross-validation.
- Evaluation using aggregate metrics, hard-scene subset metrics, and false-positive suppression diagnostics.
- Failure-case visualization for interpretability.

The work does not include video-based temporal modeling, LiDAR or radar fusion, multi-camera sensor fusion, full autonomous driving planning, or retraining of the YOLO detector.

1.5 Contributions

Unlike end-to-end context-aware detectors that modify the detection architecture itself, the proposed approach introduces a lightweight post-detection graph-based confidence refinement framework that can be attached to an existing detector without retraining the detector, which is the central novelty of this dissertation.

1. A complete YOLO-to-graph pipeline was implemented for driving-scene object detections.
2. A lightweight GCN-based confidence refinement module was developed without changing the YOLO architecture.
3. The mid-semester pseudo-label limitation was addressed by introducing ground-truth IoU-based supervision using BDD100K annotations.
4. Semantic context was added through class-pair interaction rules and evaluated separately from the learned GCN component.
5. The system was evaluated on a curated 500-image BDD100K subset using five-fold cross-validation.
6. False-positive suppression and failure-case analysis were included to interpret refinement behaviour beyond aggregate metrics.

1.6 Organization of the Report

The remainder of the report spans six chapters. Chapter 1 sets out the motivation, problem, aims, objectives, scope, and contributions. Chapter 2 surveys related work on object detectors, YOLO, DETR, graph neural networks, and graph-based scene perception for driving. Chapter 3 lays out the methodology, and Chapter 4 covers the implementation. Chapter 5 reports and discusses the experimental findings, and Chapter 6 closes with conclusions and directions for future work.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview

Object detection ranks among the core problems in computer vision, and it underpins the perception stack of autonomous vehicles. A perception system must detect vehicles, pedestrians, cyclists, traffic signs, traffic lights, and other road users before planning or control decisions can be made.

For autonomous driving, detection accuracy alone is not sufficient. The system must also be reliable in difficult conditions such as dense traffic, partial occlusion, night scenes, low visibility, and crowded pedestrian environments. In such situations, the interpretation of one object often depends on surrounding objects.

2.2 Object Detection in Autonomous Driving

Object detection refers to identifying object categories and localizing them using bounding boxes in an image. In autonomous driving, detector outputs are passed to later modules such as tracking, behaviour prediction, path planning, and decision-making.

Driving scenes are challenging because they contain objects of different sizes, scales, and visibility levels. Vehicles may appear in large numbers, pedestrians may be partially occluded, and traffic signs can span only a tiny fraction of the frame. Weather, lighting, motion blur, and camera viewpoint also affect detection quality.

Object detection research has followed two broad design directions. Two-stage pipelines, exemplified by Faster R-CNN [12], generate candidate regions before classifying and refining them, which tends to yield higher accuracy at greater computational cost. Single-stage pipelines, including SSD [13] and the YOLO family, instead regress boxes and classes in one pass, sacrificing some accuracy for a considerable speed advantage, which is why single-stage detectors are preferred for real-time autonomous driving perception. Large-scale benchmarks such as MS COCO [14] have been important in this progress, as they provide the common object categories and evaluation protocol used to train and compare detectors such as YOLO before they are adapted to driving-specific datasets such as BDD100K.

The BDD100K dataset [4] is used in this dissertation because it was specifically collected to reflect this kind of real-world driving diversity. It contains images captured across different times of day, weather conditions, and traffic densities, and provides object-level annotations for the driving-relevant categories used in this work. This makes it more representative of complex, safety-critical driving scenarios than smaller or more homogeneous detection datasets, and better suited to evaluating whether contextual refinement helps in exactly the hard-scene conditions that motivate this dissertation.

2.3 YOLO-Based Object Detection

The YOLO series of detectors [1] is a common choice for real-time detection. It casts detection as a single-stage regression problem in which box coordinates and class probabilities are produced together in one forward pass through the network.

YOLO has evolved considerably since its first version. Early versions established the single-pass detection paradigm but were comparatively weaker on small and overlapping objects. Later versions progressively improved backbone feature extraction, multi-scale detection heads, and training strategies such as improved anchor handling and loss functions, which increased accuracy while keeping inference fast enough for real-time use. YOLOv8, used in this dissertation, is part of this evolution and adopts an anchor-free detection head together with a more efficient backbone/neck design, which improves the trade-off between speed and accuracy compared with earlier YOLO versions.

In this dissertation, YOLOv8n [5][11] is used as the baseline detector. The model is kept frozen throughout the implementation. YOLO's output for each detection comprises a bounding box, a predicted class, and an associated confidence value. These confidence scores form the base scores that are later refined by the graph-based module.

2.4 Transformer-Based Object Detection

DETR [2] and related transformer-based detectors reframe detection as predicting an unordered set of objects, using attention so the model can weigh relationships between different regions of the image.

In practice, the original DETR formulation suffers from slow training convergence and struggles with small objects — a concern for driving scenes that often contain distant pedestrians or signage. Deformable DETR [15] mitigates this by restricting attention to a compact, learned set of sampling points near a reference location, rather than scanning the entire image, which speeds up convergence and improves small-object detection. This confirms that even transformer-based detectors benefit from focusing attention on a structured, relevant subset of the scene, which is conceptually related to how the graph-based refinement in this dissertation restricts contextual reasoning to nearby detections rather than the whole image.

DETR remains relevant here because it illustrates how much global scene context can contribute to detection quality. However, this dissertation follows a different direction by keeping YOLO as the baseline and adding an explicit graph-based refinement layer after detection.

2.5 Graph Neural Networks

Graph Neural Networks operate directly on graph-structured inputs, defined by a set of nodes and the edges connecting them. In this dissertation, each detected object is represented as a node, and relationships between nearby objects are represented as edges. As surveyed comprehensively by Wu et

al. [16], GNN variants differ mainly in how they aggregate neighbourhood information and range from spectral convolution-based methods to attention-based and spatial message-passing methods.

Graph Convolutional Networks [3] aggregate information from neighbouring nodes using a fixed, degree-normalized aggregation rule. This makes GCNs suitable for relational reasoning tasks where the meaning of an entity depends on its neighbourhood, and their aggregation rule is simple and computationally lightweight, which is one reason GCNs are used in this dissertation rather than a heavier architecture. Graph Attention Networks (GAT) [17] extend this idea by learning attention weights over neighbours instead of using fixed normalization, allowing the model to weight some neighbouring detections more heavily than others. GAT-style attention is noted here as a natural extension of the present GCN formulation and is discussed further as future work in Chapter 6.

Intuitively, a GCN layer works through message passing: every node gathers the feature vectors held by its neighbours, aggregates them (typically through a weighted aggregation such as averaging over the neighbourhood), and refreshes its own representation from this aggregated signal before a non-linear activation is applied. With two stacked layers, as used here, a node first absorbs information from its direct neighbours and then, through the second layer, from its neighbours' neighbours as well. Applied to a detection graph, this means the refined representation of one detected object is shaped by the geometry, confidence, and class information of nearby detections, which is precisely the contextual signal that a per-object detector such as YOLO does not model explicitly.

2.6 Scene Graphs and Context-Aware Perception

A scene graph represents a scene in terms of objects and their relationships. In autonomous driving, scene graph reasoning is useful because vehicles, pedestrians, traffic signs, and signals appear in meaningful spatial and semantic arrangements [6][7][8][9]. Early work on scene graph generation, such as the iterative message-passing formulation of Xu et al. [18], showed that jointly refining object and relationship predictions through repeated message passing between object and relationship nodes improves both detection and relationship prediction over treating them independently. A more recent survey by Zhu et al. [19] consolidates this line of work and highlights a general trend toward using scene graphs not just for descriptive scene understanding but as an intermediate representation that downstream tasks, including detection refinement, can consume.

Recent work on scene graphs in autonomous driving has used such relational structures for tasks including interaction-aware planning and topology reasoning at intersections and traffic-control points [7][8][9]. The common theme across this line of work is that treating a scene as a set of independent objects discards information that is only visible when relationships are modeled explicitly, such as which vehicles are yielding to which pedestrians, or which traffic light governs which lane. This dissertation adopts the same underlying idea at a smaller scale: rather than reasoning about full driving behaviour, it uses relational structure only to refine per-object confidence scores.

The present dissertation does not attempt full scene graph generation or end-to-end driving control. Instead, it focuses on a lightweight post-detection task: refining detection confidence scores using graph context.

2.7 Semantic Context in Object Detection

Spatial proximity alone may not fully describe the importance of object relationships. Two objects may be close, but their relationship may not be equally meaningful. For example, a person-car relationship has more safety relevance than a sign-sign relationship [6].

The final implementation includes semantic context through class-pair relationships. This allows the experiment to separate the original YOLO confidence, the learned correction from the GCN, and the hand-designed semantic context rule.

2.8 Research Gap

The reviewed literature shows that object detection has achieved strong performance through deep learning models such as YOLO [1] and DETR [2]. It also shows that graph-based reasoning and scene graphs [7][8][9] are useful for modeling object relationships in complex scenes.

Prior work has established that both scene-wide context and object-to-object relationships can sharpen detection outcomes. Structure Inference Net [20], for instance, treats detection itself as a graph-structure-inference task, embedding objects as nodes and their relationships as edges inside the detector. More recent autonomous driving work, such as GraphRelate3D [21], extends relationship-graph reasoning to 3D object detection, again by modifying how the detector uses inter-object relationships. However, many such methods either modify the detector architecture, operate in 3D perception settings, or focus on end-to-end planning and topology reasoning [7][8][9]. The gap addressed in this dissertation is narrower and more modular: using a frozen YOLO detector and applying a lightweight graph-based post-detection module only to refine confidence scores, without altering the detector or requiring 3D sensor input.

Table 2.1: Positioning of the Proposed Work Relative to Related Methods

Study	Method	Limitation Relative to This Work
YOLO [1]	CNN-based single-stage detection	No explicit inter-object context
DETR [2] / Deformable DETR [15]	Transformer set-prediction	Higher complexity; context is implicit in attention
Structure Inference Net [20]	Graph structure inference inside the detector	Requires modifying the detector architecture
GraphAD [8]	Scene graph for driver-interaction modelling	Targets end-to-end planning rather than per-detection confidence
T2SG [9]	Scene graph for traffic-topology reasoning	Targets lane/intersection topology, not per-detection confidence

Study	Method	Limitation Relative to This Work
Proposed Work	Frozen YOLO + post-detection GCN refinement	Lightweight, modular contextual confidence refinement

There remains a gap for a lightweight and modular refinement framework that can be attached after a standard object detector. This dissertation addresses the gap by combining a frozen YOLOv8n detector with graph construction, GCN-based confidence refinement, semantic rule-based ablation, and ground-truth IoU-based evaluation on BDD100K [4].

It is worth clarifying why this is framed as post-detection refinement rather than detector retraining. Retraining or fine-tuning YOLO end-to-end could, in principle, also let the model learn to use contextual cues, but it would couple the contextual reasoning tightly to the detector architecture, require re-running detector training whenever the context model changes, and make it harder to isolate how much of any performance change is due to context versus other retraining effects. Keeping YOLO frozen and adding a separate graph-based refinement stage keeps the two concerns separate: detection quality remains attributable to YOLO alone, and any change in confidence behaviour can be attributed specifically to the graph-based module. This separation is also what makes the ablation study in Chapter 5, comparing YOLO baseline, GCN refinement, semantic rule refinement, and their combination, meaningful.

2.9 Summary

This chapter has surveyed the background relevant to the dissertation. YOLO provides a fast and practical object detection baseline, but its final confidence scores do not explicitly encode object relationships. Graph Neural Networks are well matched to this need for relational reasoning, since they can explicitly capture interactions among detected objects.

CHAPTER 3: METHODOLOGY

3.1 Methodology Overview

The methodology adopted here refines detections after the fact, in a modular way, rather than altering the detector itself. YOLO is retained unmodified as the baseline, and graph-based reasoning is layered on top of its outputs.

- YOLO baseline detection
- Scene graph construction from YOLO detections
- Ground-truth IoU-based labeling
- GCN-based confidence refinement
- Evaluation and ablation analysis

3.2 Overall Pipeline

The same detection set is used for all methods. The difference lies only in the confidence score assigned to each detection. Figure 3.1 summarizes the end-to-end pipeline, from raw driving-scene images through YOLO detection, graph construction, GCN-based confidence refinement, and evaluation.

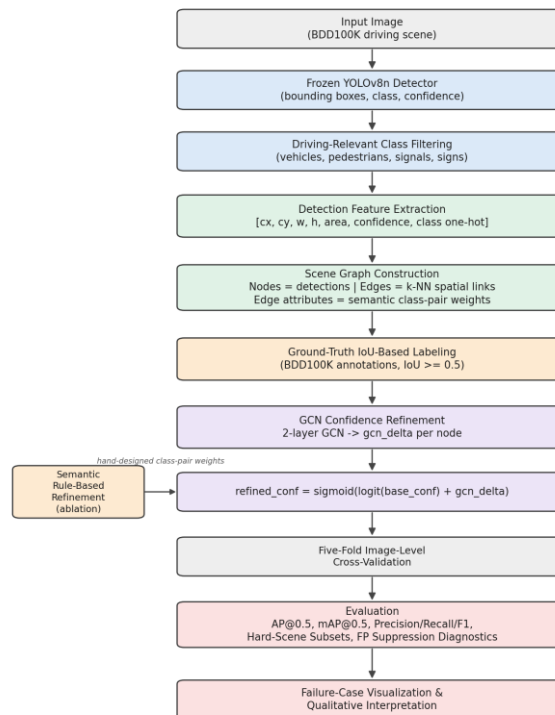


Figure 3.1: End-to-End YOLO-to-Graph-to-GCN Refinement Pipeline

Source: Author's illustration

Table 3.1: Ablation Variants

Method	Description
YOLO baseline	Uses the original YOLO confidence score
YOLO + GCN learned refinement	Uses YOLO confidence plus learned GCN correction
YOLO + semantic rule refinement	Uses YOLO confidence plus hand-designed semantic context adjustment
YOLO + GCN + semantic rule refinement	Combines learned GCN correction with semantic rule adjustment

3.3 YOLO Baseline Detection

YOLOv8n [5][11] is used as the baseline object detector. The model is used exactly as pretrained on the 80-class MS COCO vocabulary [14]; it is not retrained or fine-tuned on BDD100K, and is kept frozen throughout the experiment. It is applied only in inference mode to generate initial object detections on BDD100K images. For each input image, YOLO produces bounding box coordinates, a COCO class label, and a class confidence score.

Because YOLO's 80 COCO classes include many categories that never meaningfully occur in driving scenes (for example, household or kitchen objects), only the COCO categories relevant to driving are kept: person, bicycle, car, motorcycle, bus, truck, traffic light, and the traffic-sign/stop-sign and train classes. Any YOLO detection predicted as a non-driving-relevant COCO class is discarded at this stage. This filtering is applied to YOLO's own predictions, not to the BDD100K dataset itself; the corresponding BDD100K ground-truth category names are separately mapped onto this same driving-relevant label set so that predicted and ground-truth classes can be compared consistently during the IoU-based labeling described in Section 3.7.

3.4 Detection Feature Extraction

Each YOLO detection is converted into a feature vector. The basic geometric and confidence features are $[cx, cy, w, h, area, confidence]$ — cx and cy denote the box's normalized centre coordinates, while w and h give its normalized width and height, $area$ is $w \times h$, and $confidence$ is the YOLO detection confidence.

In the final ground-truth-supervised graph construction, class information is also added using one-hot encoding for the selected driving-relevant classes.

3.5 Scene Graph Construction

Each image is converted into a graph. Nodes represent detected objects, node features store geometry, confidence, and class information, edges represent relationships between nearby objects, and edge attributes store semantic interaction weights. Spatial edges are created using k-nearest-neighbour relationships between object centers.

More formally, an image maps to a graph $G = (V, E)$, in which V holds one node per detection and E links detections that lie spatially close to one another. Each node i in V carries a feature vector x_i , defined as:

$$x_i = [cx_i, cy_i, w_i, h_i, area_i, conf_i, onehot(class_i)] \quad (3.1)$$

Here, cx_i and cy_i give the normalized centre of the box; w_i and h_i are its normalized width and height; $area_i$ is the product $w_i \times h_i$; $conf_i$ is YOLO's confidence value; and $onehot(class_i)$ one-hot-encodes the detection's class among the driving-relevant categories.

3.6 Semantic Context Modeling

Spatial proximity alone is not enough to describe object relationships in driving scenes. To include semantic information, class-pair interaction weights are assigned based on the object classes connected by an edge [6].

Table 3.2: Semantic Context Examples

Object Pair	Contextual Meaning	Relative Weight
person-car	pedestrian near vehicle	high
person-truck	pedestrian near large vehicle	high
car-traffic light	traffic-control context	medium/high
car-traffic sign	road-rule context	medium
default pair	weak or unknown relation	low

3.7 Ground-Truth IoU-Based Labeling

The mid-semester version used pseudo-labels derived from YOLO confidence. The final methodology uses ground-truth IoU-based labels from BDD100K [4] annotations. A detection counts as a true positive when its predicted class agrees with a still-unmatched ground-truth object's class and the IoU between them reaches at least 0.5; any detection failing this test is labelled a false positive.

3.8 GCN-Based Confidence Refinement

Every graph-convolution layer refreshes a node's representation by pooling information from its neighbours. Adopting the standard GCN update rule [3], this is written as:

$$H^{(l+1)} = \sigma(\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2} H^{(l)} W^{(l)}) \quad (3.2)$$

$\hat{A} = A + I$ denotes the adjacency matrix with self-loops added, $\hat{D}$ is its degree matrix, $H^{(l)}$ collects the node representations at layer l ($H^{(0)}$ equals X , the feature matrix of Equation 3.1), $W^{(l)}$ is a trainable weight matrix, and σ is a non-linearity — ReLU, for the hidden layer used here.

The GCN receives the scene graph as input and produces one scalar output for each node. This scalar is treated as a learned confidence correction, called gcn_delta . The confidence refinement is performed in logit space:

$$refined_conf_i = \sigma_i(\text{logit}(conf_i) + gcn_delta_i) \quad (3.3)$$

where σ_i denotes the sigmoid function and $\text{logit}(p) = \ln(p / (1 - p))$. Performing the correction in logit space, rather than adding gcn_delta directly to $conf_i$, keeps the refined confidence value bounded within (0, 1) regardless of the magnitude of the correction.

A positive GCN delta increases detection confidence, while a negative delta decreases it. This is important because the refinement model should be able to both boost likely true detections and suppress likely false positives.

3.9 Semantic Rule-Based Refinement

A semantic rule-based refinement is included as an ablation. This rule adjusts confidence scores based on manually defined class-pair relationships. It is included to test whether simple semantic context alone can influence detection confidence.

3.10 Cross-Validation Methodology

Five-fold cross-validation was used for the final evaluation, with folds split by image rather than by individual detection. For each fold, approximately 80% of valid graph samples are used for training and approximately 20% are held out for validation.

3.11 Evaluation Metrics

The primary ranking-based metrics are AP@0.5 and mAP@0.5. Threshold-based metrics include precision, recall, and F1-score. Additional analyses include hard-scene subset evaluation, false-positive suppression diagnostics, and qualitative failure-case visualization.

3.12 Hard-Scene Subset Evaluation

Table 3.3: Curated 500-Image Dataset Distribution

Selection Category	Number of Images
pedestrian_vehicle	125
dense_traffic	125
occlusion_like	100
night_low_visibility	75
traffic_control	50
general_driving	25
Total	500

3.13 Summary

This chapter has set out the dissertation's methodology: YOLOv8n is kept frozen, its detections are recast as graph-structured data, a GCN is trained against ground-truth IoU-based labels, and the resulting learned corrections adjust YOLO's confidence scores.

CHAPTER 4: IMPLEMENTATION

4.1 Overview

This chapter sets out how the proposed graph-based, context-aware refinement framework was implemented. The final implementation follows the pipeline: YOLO inference, detection filtering, scene graph construction, ground-truth IoU labeling, GCN training, confidence refinement, ablation evaluation, hard-scene analysis, and failure-case visualization.

4.2 Development Environment

The implementation was carried out in Google Colab. Google Drive was used for storing datasets, generated detections, graph files, output tables, and visualizations. The implementation was kept modular so that YOLO inference, graph construction, GCN training, evaluation, and visualization could be run independently.

Table 4.1: Development Environment

Tool / Library	Purpose
Python	Main programming language
Google Colab	Development and GPU execution environment
Google Drive	Storage of dataset, outputs, and artifacts
Ultralytics YOLO [5][11]	YOLOv8n object detection
PyTorch	Deep learning model implementation
PyTorch Geometric [10]	Graph data representation and GCN implementation
NumPy / pandas	Numerical and tabular result processing
scikit-learn	k-nearest-neighbour graph construction and cross-validation
matplotlib / PIL	Visualization of detections and failure cases

4.3 Dataset Preparation

The final experiment used a curated 500-image subset from BDD100K [4]. The dataset was selected to include complex driving scenarios rather than only simple road scenes. The selected categories were `pedestrian_vehicle`, `dense_traffic`, `occlusion_like`, `night_low_visibility`, `traffic_control`, and `general_driving`.

4.4 YOLO Inference Pipeline

YOLOv8n was used as the baseline detector. The pretrained YOLOv8n model was loaded using the Ultralytics library [5] and kept frozen. For each image, YOLO generated bounding box coordinates, predicted class label, and confidence score. The detections were saved into a structured JSON file.

4.5 Driving-Relevant Class Filtering

YOLOv8n is pretrained on the 80-class MS COCO vocabulary, which includes many categories that are not relevant to driving scenes (for example, household or kitchen objects). The implementation

therefore keeps only the COCO categories aligning with the driving-relevant label set used for BDD100K: person, bicycle, car, motorcycle, bus, truck, traffic light, train, and stop sign, with the last of these serving as COCO's nearest equivalent to BDD100K's traffic sign category and relabelled accordingly. All other COCO classes are discarded at this stage.

On the ground-truth side, BDD100K's own category set is mapped onto the same nine-class evaluation ontology so that predicted and ground-truth classes can be compared consistently; in particular, BDD100K's rider category, which has no direct COCO equivalent, is merged into person for this purpose. This mapping is applied once and reused for both the IoU-based labeling in Section 4.8 and the hard-scene evaluation in Chapter 5.

4.6 Scene Graph Construction

After YOLO detections were generated and filtered, each image was converted into a scene graph. Each graph contains nodes representing detected objects, node features representing detection attributes, edges representing object relationships, and edge attributes representing semantic relationship weights. From the 500 selected images, 499 valid graph samples were produced.

4.7 Semantic Edge Attributes

Semantic class-pair information was added to the graph because spatial closeness alone does not fully represent the importance of relationships in driving scenes. Semantic weights were assigned based on object class pairs and stored as edge attributes.

4.8 Ground-Truth IoU-Based Graph Labeling

The final implementation replaced pseudo-labels with ground-truth IoU-based labeling using BDD100K annotations. A detection was marked true positive when it matched a still-unmatched ground-truth box of the same class at $\text{IoU} \geq 0.5$, and false positive otherwise.

4.9 GCN Model Implementation

The refinement model itself is a two-layer GCN [3]: the first layer projects node features into a 32-dimensional hidden space and applies ReLU. The second layer outputs one scalar confidence correction per node.

4.10 Training Setup

The GCN was trained using ground-truth IoU-based node labels. The task is binary classification of detection nodes as true positives or false positives. BCEWithLogitsLoss and the Adam optimizer were used. Training was performed independently inside each cross-validation fold.

4.11 Cross-Validation Implementation

Five-fold cross-validation, split at the image (graph) level rather than per detection, was used for the final experiment. Predictions from all folds were later combined to compute pooled out-of-fold metrics.

4.12 Ablation Variants Implemented

Four confidence variants were evaluated: YOLO baseline, YOLO + GCN learned refinement, YOLO + semantic rule refinement, and YOLO + GCN + semantic rule refinement.

Table 4.2: Output Artifacts

Output	Purpose
detections.json	Stores YOLO detection results
experiment_config.json	Stores run configuration
Graph .pt files	Stores PyTorch Geometric graphs
GT-labeled graph files	Stores graphs with IoU-based node labels
Cross-validation CSV files	Stores fold-wise AP/mAP results
Operating-point summary CSV	Stores precision, recall, and F1 at threshold 0.25
Hard-scene metrics CSV	Stores subset-level evaluation results
False-positive diagnostic CSV	Stores suppression and boosting statistics
Failure-case visualization images	Stores qualitative examples

4.13 Final System Configuration Summary

Table 4.3 summarizes the final end-to-end configuration used to produce the results reported in Chapter 5. It is included as a quick-reference summary of the settings that define the reported system, consolidating details that are otherwise spread across this chapter.

Table 4.3: Final System Configuration Summary

Item	Final Setting
Detector	YOLOv8n (frozen)
Dataset	BDD100K, curated 500-image subset
Valid graph samples	499
Node features	Geometry + confidence + class one-hot
Graph edges	k-NN spatial edges with semantic class-pair edge attributes
Supervision	Ground-truth IoU-based labeling (IoU $\geq$ 0.5)
Refinement model	2-layer Graph Convolutional Network (GCN)
Evaluation protocol	Five-fold image-level cross-validation

4.14 Implementation Summary

The final implementation successfully converted the proposed methodology into an executable pipeline. The system starts with YOLOv8n detections, constructs graph-structured scene representations, labels detection nodes using BDD100K ground truth, trains a GCN for confidence refinement, and evaluates multiple confidence refinement variants. The next chapter presents the results obtained from this implementation.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Overview

The final evaluation was performed on a curated 500-image subset of BDD100K. Out of these 500 selected images, 499 images produced valid graph samples after YOLO detection filtering and graph construction. The evaluation studies overall cross-validation results, pooled out-of-fold evaluation, fixed-threshold operating point analysis, hard-scene subset behavior, false-positive suppression, and failure cases.

5.2 Dataset Summary

Table 5.1: Curated 500-Image Dataset Profile

Scene Category	Number of Images
pedestrian_vehicle	125
dense_traffic	125
occlusion_like	100
night_low_visibility	75
traffic_control	50
general_driving	25
Total selected images	500

Table 5.2: Detection and Graph Labeling Summary

Item	Count
Selected images	500
Valid graph samples	499
Raw YOLO detections	12,244
Driving-relevant detections	12,151
Ground-truth objects	19,895

5.3 Five-Fold Cross-Validation Results

Table 5.3: Five-Fold Cross-Validation Results

Method	AP@0.5	mAP@0.5
YOLO baseline	0.2356 +/- 0.0039	0.1668 +/- 0.0180
YOLO + GCN learned refinement	0.2354 +/- 0.0043	0.1667 +/- 0.0183
YOLO + semantic rule refinement	0.2358 +/- 0.0041	0.1669 +/- 0.0183
YOLO + GCN + semantic rule refinement	0.2355 +/- 0.0045	0.1670 +/- 0.0185

The results show that the proposed graph-based refinement methods remain very close to the YOLO baseline in terms of AP@0.5 and mAP@0.5. The semantic rule refinement gives a very small AP

increase, and the hybrid method gives a very small mAP increase, but the differences are too small to claim a strong global improvement.

The correct interpretation is that the final model does not clearly outperform YOLO on aggregate ranking-based metrics. However, the low standard deviation across folds shows that the 500-image experiment is more stable than the earlier small-run experiment.

5.4 Comparison with Earlier Small-Run Experiment

An earlier small-run experiment was used during development to verify the pipeline. That experiment showed higher variance across folds because the number of images was very small. After scaling to the curated 500-image subset, the cross-validation results became more stable.

The AP@0.5 standard deviation reduced from approximately 0.107 in the smaller run to approximately 0.004 in the 500-image run. This indicates that the larger experiment provides a more reliable estimate of model behaviour.

As an additional robustness check, the regularized variant of the GCN (dropout of 0.3, combined with early stopping on held-out fold loss) was also trained across all five folds to test whether the near-baseline result was mainly caused by overfitting. Figure 5.1 shows the resulting train versus validation BCE loss curves. Train and validation losses stay close together in every fold, with no clear divergence, which indicates that overfitting is not the primary explanation for the limited AP/mAP improvement. Dropout and early stopping did not materially change the conclusion: refined scores remained close to the YOLO baseline, with differences small compared with fold-to-fold variation.

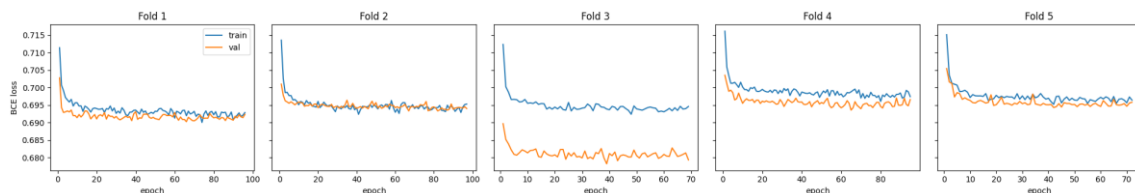


Figure 5.1: Train vs Validation BCE Loss Curves Across Five Folds (Regularized GCN)

Source: Author’s experiment output (Phase 7E regularization diagnostic)

5.5 Pooled Out-of-Fold Evaluation

Table 5.4: Pooled Out-of-Fold AP@0.5 Results

Method	Pooled AP@0.5
YOLO baseline	0.2348
YOLO + GCN learned refinement	0.2345
YOLO + semantic rule refinement	0.2350
YOLO + GCN + semantic rule refinement	0.2346

The pooled results support the same conclusion as the fold-wise cross-validation table. All methods remain close to the YOLO baseline. The semantic rule refinement is slightly higher than YOLO, but the margin is very small.

5.6 Fixed-Threshold Operating Point Analysis

Table 5.5: Fixed-Threshold Results at Confidence Threshold 0.25

Method	F1-score
YOLO baseline	0.3203
YOLO + GCN learned refinement	0.3176
YOLO + semantic rule refinement	0.3281
YOLO + GCN + semantic rule refinement	0.3263

At the fixed threshold of 0.25, the semantic rule refinement and hybrid refinement show small F1-score improvements over the YOLO baseline. The improvement is mainly due to increased recall, although precision reduces in some cases. This result should be interpreted as an operating-point effect rather than a strong AP/mAP improvement.

5.7 Hard-Scene Subset Analysis

Table 5.6: Representative Hard-Scene F1 Improvements

Scene Subset	Best Observed F1 Improvement
occlusion_like	approximately +0.0117
night_low_visibility	approximately +0.0098
pedestrian_vehicle	approximately +0.0079
dense_traffic	approximately +0.0049
city street	approximately +0.0079

The results show small threshold-level F1 improvements in selected hard-scene subsets. The strongest gains appear in occlusion-like and night/low-visibility categories. Pedestrian-vehicle scenes also show a small positive effect. These results support a cautious conclusion: graph-based context refinement shows useful behavior in selected difficult scene categories, but the current lightweight feature representation is not strong enough to produce a consistent global improvement.

5.8 False-Positive Suppression Analysis

Table 5.7: False-Positive Suppression Diagnostic

Method	FP Suppressed	FP Boosted	Mean FP Delta
YOLO + GCN learned refinement	1409	969	-0.002984
YOLO + semantic rule refinement	0	2006	+0.016418

Method	FP Suppressed	FP Boosted	Mean FP Delta
YOLO + GCN + semantic rule refinement	420	2097	+0.013098

The learned GCN refinement suppresses a large number of false positives and has a negative mean false-positive delta. This means that the GCN is not simply increasing all confidence values. It is learning to reduce confidence for some detections that are not supported by contextual evidence.

The semantic rule refinement behaves differently. It mainly boosts confidence and does not suppress false positives. This can improve recall but can also increase false positives. The hybrid method inherits some suppression ability from the GCN, but the semantic rule component still dominates the confidence changes.

5.9 Failure-Case Visualization

In addition to the aggregate and threshold-level metrics above, YOLO baseline and GT-supervised refined detections were compared directly on individual images to inspect qualitative refinement behaviour. Figure 5.2 and Figure 5.3 show two representative driving scenes, with YOLO baseline detections on the left and GCN-refined detections on the right at a display threshold of 0.25. The refined boxes retain the same coordinates and class labels as YOLO; only the confidence values and, consequently, which boxes clear the display threshold, change.

GT-supervised CV qualitative result: bb97c366-2adf3cdc.jpg

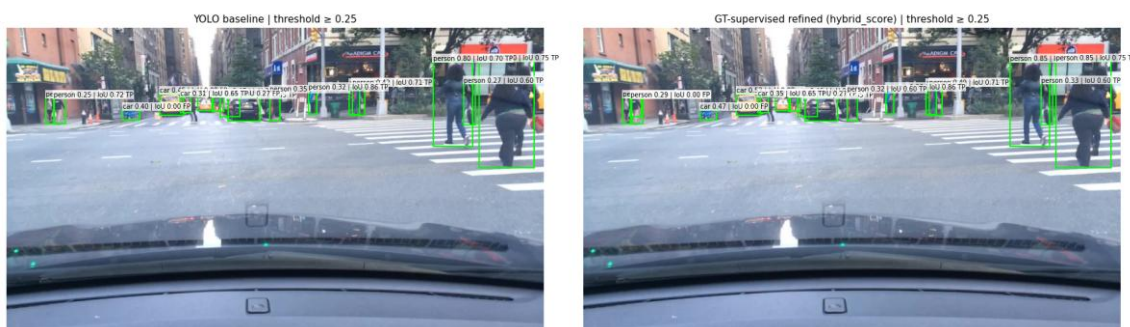


Figure 5.2: Qualitative Comparison of YOLO Baseline and GT-Supervised Refined Detections – Example A

Source: Author's experiment output (Phase 8A qualitative inspection)

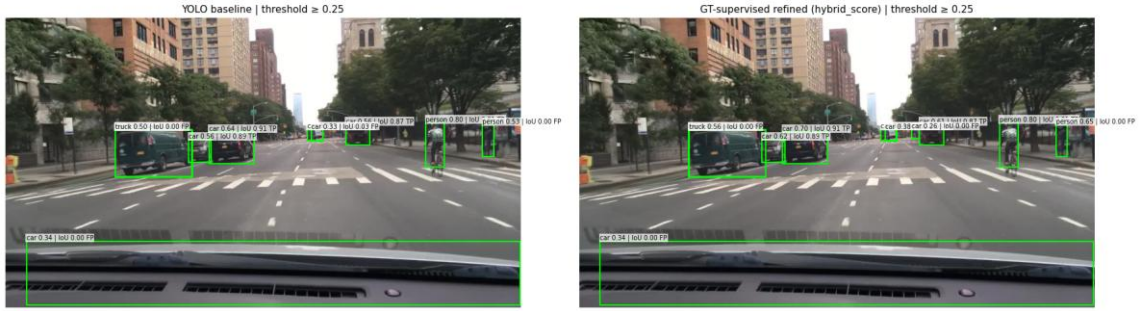


Figure 5.3: Qualitative Comparison of YOLO Baseline and GT-Supervised Refined Detections – Example B

Source: Author's experiment output (Phase 8A qualitative inspection)

A further inspection targeted borderline detections, that is, YOLO detections that fell just below the 0.25 display/operating threshold. Figure 5.4 shows an example where a pedestrian detection with YOLO confidence 0.24 was refined to 0.34 by the GCN, moving it above the threshold. This kind of case illustrates the practical value of confidence refinement independently of the aggregate AP/mAP result: even without a global ranking improvement, the refinement can rescue individual borderline detections that would otherwise be discarded at a fixed operating threshold.

Phase 8B best-case example: rescued_crossed_threshold
c4d25f00-3b1d8669.jpg | person | YOLO 0.24 → refined 0.34 | Δ +0.10



Figure 5.4: Borderline Detection Crossing the Confidence Threshold After Refinement

Source: Author's experiment output (Phase 8B best-case qualitative selection)

Table 5.8: Failure-Case Visualization Categories

Case Type	Number of Examples
False positives boosted	4
True positives suppressed	4
False positives suppressed	2
True positives boosted	2
Total	12

The visualizations were intentionally selected to include both success and failure cases. This avoids presenting only favourable examples and helps explain why AP/mAP does not improve strongly even though some useful context-aware behavior is present. Figure 5.5 illustrates one of the two “true positives boosted” cases from Table 5.8: a person detection with YOLO confidence 0.40 is refined upward to 0.54 while its bounding box and class label are unchanged. The remaining ten examples in Table 5.8, including the false-positive-boosted and true-positive-suppressed cases, are retained as supporting evidence for the discussion below but are not all reproduced here to keep the chapter concise.

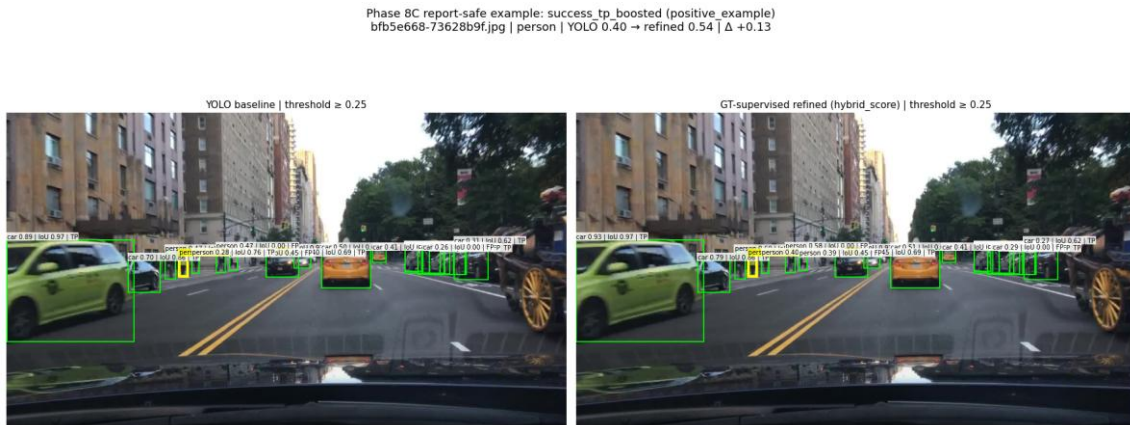


Figure 5.5: Example of True-Positive Confidence Boosting (Report-Safe Case)

Source: Author’s experiment output (Phase 8C report-safe case selection)

5.10 Evaluator Feedback Resolution

Table 5.9 summarizes how each concern raised at the mid-semester evaluation was addressed in the final implementation, tying the changes described throughout this report back to that feedback.

Table 5.9: Evaluator Feedback Resolution

Feedback Item	Mid-Semester Status	Final Status
Pseudo-label circularity	Present	Resolved – ground-truth IoU-based labeling (Section 3.7)
Ground-truth supervision	Missing	Added – BDD100K annotation-based labels
Hard-scene analysis	Missing	Added – Section 5.7
False-positive suppression evidence	Not demonstrated	Demonstrated – Section 5.8
Failure-case analysis	Missing	Added – Section 5.9

5.11 Discussion

The final results show that the proposed YOLO + GCN refinement pipeline was successfully implemented and evaluated. The system converts YOLO detections into graph samples, trains a GCN using ground-truth IoU labels, and produces refined confidence scores.

The AP@0.5 and mAP@0.5 results remain close to YOLO. This means that the current graph-based refinement does not produce a stable global ranking improvement over the detector baseline. This is an important result and is reported honestly.

There are several reasons why the improvement may be limited. The current node representation is lightweight: it includes the discrete class label that YOLO already predicts (for example, whether a detection is a car or a pedestrian), together with bounding box geometry and confidence, encoded as in Equation 3.1. What it does not include is the underlying visual appearance of the detected region itself, that is, the pixel-level texture, colour, and fine-grained shape information inside the bounding box. Knowing that a box is labeled 'car' is not the same as the GCN being able to see what that particular car crop looks like. Also, if YOLO misses an object completely, the GCN cannot recover it because there is no node corresponding to the missed object.

Despite these limitations, the final results are useful. The 500-image experiment confirms that the pipeline is feasible and stable. The learned GCN demonstrates false-positive suppression behaviour, and hard-scene subset analysis shows small F1 gains in selected complex driving conditions. Thus, the dissertation outcome is best understood as a validated implementation and diagnostic study of graph-based confidence refinement, rather than a claim of strong YOLO performance improvement.

5.12 Key Findings

1. The proposed YOLO-to-graph-to-GCN pipeline was successfully implemented.
2. The mid-semester pseudo-label circularity was removed using BDD100K ground-truth IoU-based labels.
3. The final experiment used a curated 500-image BDD100K subset, producing 499 valid graph samples.
4. Overall AP@0.5 and mAP@0.5 remained very close to the YOLO baseline.
5. At a fixed threshold of 0.25, semantic rule and hybrid refinement showed small F1-score gains over YOLO.
6. Hard-scene subset analysis showed small F1 gains in selected difficult scenes.
7. The learned GCN suppressed 1409 false-positive detections, showing genuine negative-delta behavior.
8. Failure-case visualization showed both successful and unsuccessful refinement examples, including a regularization diagnostic (Figure 5.1) confirming the near-baseline result is not driven by overfitting.

5.13 Threats to Validity

As with any single-study evaluation, the results in this chapter should be interpreted alongside the following limitations of the experimental design:

- The curated 500-image subset was selected to emphasize hard-scene categories and may not represent the full distribution of BDD100K or of real-world driving conditions.
- Graph nodes use geometric, confidence, and class-label features only; the underlying visual appearance (pixel-level texture and shape) inside each detected region is not embedded, which may limit how much contextual signal the GCN can exploit.
- The overall dataset size (500 images, 499 valid graphs) is modest for deep-learning evaluation; larger-scale evaluation would give a more precise estimate of AP/mAP differences.
- Detections missed entirely by the frozen YOLO baseline cannot be recovered by the graph-based refinement, since no corresponding node exists in the graph.
- The semantic rule weights used in the rule-based ablation were manually designed rather than learned from data, so their behaviour reflects the specific rules chosen rather than an optimized semantic model.

5.14 Summary

This chapter presented the final results and discussion. The main conclusion is that the proposed graph-based refinement framework is functional and evaluable, but it does not strongly outperform YOLO on aggregate AP/mAP metrics. The most meaningful outcomes are improved evaluation rigor, replacement of pseudo-labels with ground-truth IoU labels, hard-scene analysis, and false-positive suppression behavior of the learned GCN.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Summary of Work

This dissertation has put forward a context-aware, graph-based framework for refining object-detection confidence in autonomous-driving scenes. The work was motivated by the limitation that standard object detectors such as YOLO provide fast and effective object detection, but their final confidence scores do not explicitly model relationships between neighbouring objects.

The proposed system keeps YOLOv8n as a frozen baseline detector and applies a Graph Convolutional Network after detection. Every YOLO detection becomes a node in the graph, with edges capturing the relationships among detections. The GCN processes this scene graph and produces a learned confidence correction for each detected object.

6.2 Key Findings

The final 500-image experiment showed that AP@0.5 and mAP@0.5 remained very close to the YOLO baseline. This means that the proposed method did not produce a strong global ranking improvement over YOLO.

However, fixed-threshold analysis showed small F1-score improvements for the semantic rule and hybrid refinement methods. Hard-scene subset analysis showed small F1 gains in selected difficult driving categories. The learned GCN component also demonstrated genuine false-positive suppression behavior by suppressing 1409 false-positive detections.

6.3 Contributions

The main contributions include a complete post-detection graph refinement pipeline, conversion of YOLOv8n detections into scene graphs, GCN-based confidence refinement without modifying YOLO, ground-truth IoU-based supervision, structured ablation analysis, hard-scene subset evaluation, and failure-case visualization.

6.4 Limitations

The GCN operates only on YOLO detections. If YOLO completely misses an object, the graph model cannot recover it because no node exists for that missed object. The node features are also lightweight: they use the class label YOLO already assigns, but do not include the underlying visual appearance (pixel-level texture and shape) of each detected region. Semantic rule refinement can improve recall but may also boost false positives. The experiment also uses static images only and does not include temporal modeling across video frames.

6.5 Future Work

The most important improvement is to add ROI-level visual appearance embeddings to the graph node features, which the current implementation does not use. Future work can extract visual embeddings

from cropped object regions or intermediate YOLO feature maps and add them to the node features described in Equation 3.1. This would allow the GCN to reason using both relational context and raw visual appearance, rather than context and class label alone.

Other future directions include scaling evaluation to larger BDD100K subsets, adding temporal modeling across video frames, comparing GCN with GATv2 or edge-conditioned GNNs, learning semantic relationship weights from data, testing stronger detectors, and adding deployment-oriented runtime analysis.

6.6 Computational Overhead

Since the GCN operates only on detected objects rather than on image pixels, its computational overhead is substantially lower than retraining or replacing the detector. The graph module processes only the scene graph generated from YOLO detections, typically a few dozen nodes per image, through a shallow two-layer network, so the added cost of confidence refinement is small relative to the YOLO forward pass itself.

6.7 Concluding Remarks

This dissertation successfully implemented and evaluated a graph-based context-aware confidence refinement framework for autonomous driving object detection. The final system improved upon the mid-semester version by replacing pseudo-label training with ground-truth IoU-based supervision and by adding a larger 500-image evaluation.

The results show that the proposed method does not strongly outperform YOLO in aggregate AP@0.5 or mAP@0.5. However, the work demonstrates that graph-based context refinement is feasible and demonstrates measurable contextual behavior, although a statistically significant improvement in detection ranking metrics was not observed on the current evaluation dataset. The trained GCN cuts down a substantial number of false positives, and small F1 gains at fixed thresholds appear in several of the harder scene categories. The final outcome is therefore a validated implementation and diagnostic study rather than a claim of large performance improvement.

APPENDICES

Appendix A: Dataset Selection Summary

The final experiment used a curated 500-image subset from BDD100K. Out of the 500 selected images, 499 produced valid graph samples after YOLO detection filtering and graph construction.

Appendix B: YOLO Detection Configuration

YOLOv8n serves solely as the baseline detector; the proposed method leaves its weights, bounding boxes, and class labels untouched.

Appendix C: Graph Construction Details

Each image was converted into a graph using YOLO detections. Nodes correspond to detections, edges correspond to nearby object relationships, and edge attributes store semantic class-pair information.

Appendix D: GCN Architecture

The GCN consists of two graph convolution layers. The scalar output is treated as `gcn_delta` and is combined with YOLO confidence in logit space.

Appendix E: Ablation Study Variants

The ablation study compares YOLO baseline, YOLO + GCN learned refinement, YOLO + semantic rule refinement, and YOLO + GCN + semantic rule refinement.

Appendix F: Reproducibility Notes

The implementation was carried out in Google Colab using Python, PyTorch, PyTorch Geometric, Ultralytics YOLO, NumPy, pandas, scikit-learn, matplotlib, PIL, and Google Drive. The notebook stores intermediate artifacts such as detections, graph files, cross-validation results, hard-scene metrics, and visualization outputs.

REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- [2] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-End Object Detection with Transformers,” *European Conference on Computer Vision*, 2020.
- [3] T. N. Kipf and M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” *International Conference on Learning Representations*, 2017.
- [4] F. Yu et al., “BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020.
- [5] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLO,” Available: <https://github.com/ultralytics/ultralytics>, Accessed: Jul. 2026.
- [6] Y. Wang et al., “Spatial Context-aware Object-attentional Network for Multi-label Image Classification and Detection,” *IEEE Transactions on Image Processing*, 2021.
- [7] P. E. I. Dimasi, “Scene Graph Generation in Autonomous Driving: A Neuro-symbolic Approach,” *Master’s Thesis, Politecnico di Torino*, 2023.
- [8] J. Li et al., “GraphAD: Interaction Scene Graph for End-to-end Autonomous Driving,” *International Joint Conference on Artificial Intelligence*, 2024.
- [9] X. Chen et al., “T2SG: Traffic Topology Scene Graph for Topology Reasoning in Autonomous Driving,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [10] PyTorch Geometric Documentation, “PyTorch Geometric: Graph Neural Network Library,” Available: <https://pytorch-geometric.readthedocs.io>, Accessed: Jul. 2026.
- [11] Ultralytics Documentation, “YOLOv8 Object Detection,” Available: <https://docs.ultralytics.com>, Accessed: Jul. 2026.
- [12] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” *Advances in Neural Information Processing Systems*, 2015.
- [13] W. Liu et al., “SSD: Single Shot MultiBox Detector,” *European Conference on Computer Vision*, 2016.
- [14] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” *European Conference on Computer Vision*, 2014.
- [15] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, “Deformable DETR: Deformable Transformers for End-to-End Object Detection,” *International Conference on Learning Representations*, 2021.
- [16] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, “A Comprehensive Survey on Graph Neural Networks,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021.
- [17] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph Attention Networks,” *International Conference on Learning Representations*, 2018.

- [18] D. Xu, Y. Zhu, C. B. Choy, and L. Fei-Fei, "Scene Graph Generation by Iterative Message Passing," IEEE Conference on Computer Vision and Pattern Recognition, 2017.
- [19] G. Zhu et al., "Scene Graph Generation: A Comprehensive Survey," Neurocomputing, 2023.
- [20] Y. Liu, R. Wang, S. Shan, and X. Chen, "Structure Inference Net: Object Detection Using Scene-Level Context and Instance-Level Relationships," IEEE Conference on Computer Vision and Pattern Recognition, 2018.
- [21] M. Liu et al., "GraphRelate3D: Context-Dependent 3D Object Detection with Inter-Object Relationship Graphs," IEEE International Conference on Intelligent Transportation Systems, 2024.

GLOSSARY

Term	Meaning	Page
AP@0.5	Average Precision calculated at IoU threshold 0.5	17
BDD100K	Berkeley DeepDrive dataset used for autonomous driving research	2
Confidence Refinement	Adjustment of detector confidence scores after initial detection	1
Edge	Relationship between two nodes in a graph	9
False Positive	Detection predicted by the model but not matched to a ground-truth object	9
GCN	Graph Convolutional Network	1
GNN	Graph Neural Network	1
Ground Truth	Human-annotated reference label	9
IoU	Intersection over Union, used to measure overlap between predicted and ground-truth boxes	9
mAP@0.5	Mean Average Precision across classes at IoU threshold 0.5	17
Node	A detected object represented in the graph	9
ROI	Region of Interest, usually the detected object region; not used as a feature source in the current implementation (see Chapter 6 Future Work)	24
Scene Graph	Graph representation of objects and their relationships in a scene	9
Semantic Rule	Hand-designed confidence adjustment based on object class relationships	9
True Positive	Detection correctly matched to a ground-truth object	9
YOLO	You Only Look Once, a real-time object detection model	1

FINAL CHECKLIST

Checklist of Items for the Final Dissertation / Project / Project Work Report

This checklist is to be attached as the last page of the final report. This checklist is to be duly completed, verified and signed by the student.

#	Checklist Item	Status
1	Is the final report neatly formatted with all the elements required for a technical Report?	Yes
2	Is the Cover page in proper format as given in Annexure A?	Yes
3	Is the Title page (Inner cover page) in proper format?	Yes
4(a)	Is the Certificate from the Supervisor in proper format?	Yes
4(b)	Has it been signed by the Supervisor?	Yes
5	Is the Abstract included in the report properly written within one page?	Yes
5	Have the technical keywords been specified properly?	Yes
6	Is the title of your report appropriate? The title should be adequately descriptive, precise and must reflect scope of the actual work done. Uncommon abbreviations / Acronyms should not be used in the title.	Yes
7	Have you included the List of abbreviations / Acronyms?	Yes
8	Does the Report contain a summary of the literature survey?	Yes
9	Does the Table of Contents include page numbers?	Yes
9(i)	Are the Pages numbered properly? (Ch. 1 should start on Page # 1)	Yes
9(ii)	Are the Figures numbered properly? (Figure Numbers and Figure Titles should be at the bottom of the figures)	Yes
9(iii)	Are the Tables numbered properly? (Table Numbers and Table Titles should be at the top of the tables)	Yes
9(iv)	Are the Captions for the Figures and Tables proper?	Yes
9(v)	Are the Appendices numbered properly? Are their titles appropriate?	Yes
10	Is the conclusion of the Report based on discussion of the work?	Yes
11	Are References or Bibliography given at the end of the Report?	Yes
11	Have the References been cited properly inside the text of the Report?	Yes
11	Are all the references cited in the body of the report?	Yes
12	Is the report format and content according to the guidelines? The report should not be a mere printout of a PowerPoint Presentation, or a user manual. Source code of software need not be included in the report.	Yes

Declaration by Student:

I certify that I have properly verified all the items in this checklist and ensure that the report is in proper format as specified in the course handout.

Place: Bangalore

Date: 28-07-2026

Signature of the Student: _____



Name: ANAGHA K M

ID No.: 2024AA05551